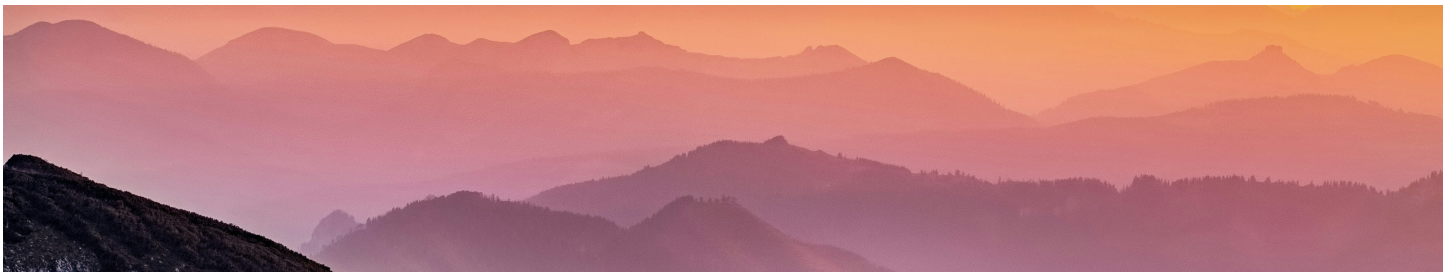

Trellix Architectural and Functional Design

Author: Issa El Mokadem

Program: 2º DAW – I.E.S. Abdera

Project Type: Full-Stack Web Application (Task Board)

Part A: Technical User Stories

This document defines the functional architecture of **Trellix.** It covers the technical user stories, entity-relationship model, and class diagram used as the foundation for implementation.

ℹ Below are the user stories, specifying events and responses in the DOM/Client.

ID	User Story (Functional)	DWEC Technical Detail (Architecture & DOM)
US-001	**As a:** Registered User **I want to:** Create a new work board **So that:** I can organize tasks in a separate space.	**Trigger:** Click "Create Board" button → CreateBoardModal opens. **Objects:** boardService.create(boardData) → Dashboard.jsx state update via handleBoardCreated(). **DOM Result:** Modal closes, new board card renders in the grid.

US-002	**As a:** Registered User **I want to:** Add a card to an existing list **So that:** I can record a pending task.	**Trigger:** Click "Add Card" inline form in ListCard component. **Objects:** useBoardStore.createCard(listId, title) in boardStore.js. **DOM Result:** New card appended to the list.
US-003	**As a:** User **I want to:** Drag a card to another list **So that:** I can visually change task status.	**Trigger:** onDragEnd event from @dnd-kit DndContext. **Objects:** moveCardOptimistic(activeId, overId) → moveCardBackend(activeId, containerId, index) via PATCH /cards/{id}/move. **DOM Result:** Card moves instantly (optimistic UI); persisted to DB on success, rolled back via fetchBoard() on failure.
US-004	**As a:** User **I want to:** View weather in sidebar. **So that:** I have external context.	**Trigger:** Sidebar WeatherWidget component, fetches on location change. **Objects:** Frontend calls om 🌤️ Free Open-Source Weather API | Open-... Open-Source ☀️ Weather API with free access for non-commercial use. No API Key required ✅. Accurate... om open-meteo.com

		No backend proxy. Location persisted in localStorage. **DOM Result:** WeatherWidget renders in the sidebar with city name, temperature (°C), weather icon, and condition label. Clicking opens a popover with search to change city..
US-005	**As an:** Admin **I want to:** Delete a user **So that:** I can manage security.	**Trigger:** Click Trash2 icon → ConfirmModal → confirm. **Objects:** adminService.deleteUser(id) in Admin.jsx. **DOM Result:** User row removed from the admin table without page reload.

⚠ Dashboard and Admin use local useState + API calls directly.

The Zustand store only manages BoardView state (lists, cards, D&D)

Part B: Entity-Relationship

ℹ Below are the different tables and their relationships.

1. **Entities and Attributes**

Entity	Attribute	Type	Description

USER	`id`	Integer	PK, unique identifier.
	`email`	String (255)	Unique, used for login.
	`username`	String (100)	Unique, visible display name.
	`password_hash`	String (255)	Bcrypt hash, never exposed via API.
	`role`	String (20)	"admin" or "user" (RBAC). Default: "user".
	`is_active`	Boolean	Soft delete / account disable flag. Default: true.
	`avatar`	String (255)	Optional profile image URL. Nullable.
	`created_at`	DateTime	Auto-generated on registration.
	`updated_at`	DateTime	
BOARD	`id`	Integer	Unique identifier.
	`title`	String (255)	Project name.
	`description`	String (1000)	Optional board description.

	`color`	String (50)	Identity color (hex). Default: #0079bf.
	`user_id`	Integer	FK → users.id. Reference to board creator.
	`created_at`	DateTime	Auto-generated on creation.
	`updated_at`	DateTime	Auto-updated on modification.
LIST	`id`	Integer	Unique identifier.
	`title`	String (255)	Column name.
	`board_id`	Integer	FK → boards.id. Reference to parent Board.
	`position`	Integer	Sort order within the board.
	`created_at`	DateTime	Auto-generated on creation.
	`updated_at`	DateTime	Auto-updated on modification.
CARD	`id`	Integer	Unique identifier.
	`title`	String (255)	Task summary.
	`description`	Text	Full task details.
	`list_id`	Integer	FK → lists.id. Reference to parent List.

	`position`	Float	Sort order within the list (supports fractional indexing for D&D).
	`priority`	String (50)	"low", "medium", or "high" — rendered as colored dot on card.
	`labels`	JSON	Array of tag strings (e.g. ["bug", "frontend"]). Rendered as chips.
	`created_at`	DateTime	Auto-generated on creation.
	`updated_at`	DateTime	Auto-updated on every modification.

2. **Relationships**

Relationship	Cardinality	Description
User - Board	**1:N**	A User creates many Boards; a Board has one owner.
Board - List	**1:N**	A Board contains many Lists.
List - Card	**1:N**	A List contains many Cards.

Part C: Class Architecture (UML)

> "SQLAlchemy data models (anemic domain model). Business logic — authentication, CRUD, card movement — lives in FastAPI endpoints and Zustand store actions, not in the model classes."

Class	Attributes (Properties)
User	• `id: Integer` • `email: String(255)` • `username: String(100)` • `password_hash: String(255)` • `role: String(20)` • `is_active: Boolean` • `avatar: String(255)` • `created_at: DateTime` • `updated_at: DateTime`

Board	`+ id: Integer`
	`+ title: String(255)`
	`+ description: String(1000)`
	`+ color: String(50)`
	`+ user_id: Integer (FK)`
	`+ created_at: DateTime`
	`+ updated_at: DateTime`

List	`+ id: Integer`
	`+ title: String(255)`
	`+ board_id: Integer (FK)`
	`+  position: Integer`

	+ created_at: DateTime + updated_at: DateTime
Card	+ id: Integer + title: String(255) + description: Text + list_id: Integer (FK) + position: Float + priority: String(50) + labels: JSON + created_at: DateTime + updated_at: DateTime

---